这份报告并不说明任何键的真实状态，驱动必须原样保留上一份。若把它当成“全部松开”，就会先上报一串假松开，下一份报告再把同样的键当作新按下，同一个字符因此重复两遍。

之后的事由输入子系统（kernel/input.c、kernel/input.h）完成。类驱动只用到七个接口：

```
struct input_dev *input_allocate_device(void);
void input_set_capability(struct input_dev *dev, int type, int code);
int  input_register_device(struct input_dev *dev);
void input_report_key(struct input_dev *dev, int code, int value);
void input_sync(struct input_dev *dev);
void input_unregister_device(struct input_dev *dev);
void input_free_device(struct input_dev *dev);
```

probe() 用前三项登记设备，运行期只用中间两项上报状态变化，remove 时用后两项收尾；事件去重、按键状态、环形队列、阻塞读、SYN_REPORT 和 SYN_DROPPED 全部由 core 负责，usbkbd.c 里只剩 USB 与 HID 的职责。core 再把事件分发给两个“处理者”：

- **evdev**：/dev/input/event0。每次 open 都分配一个独立的客户端环形缓冲区（fops.open 设置 f->private_data），读者在自己的缓冲区上睡眠；溢出时插入 SYN_DROPPED；
- **控制台**：把按键转成字符（处理 Shift、Caps Lock、Ctrl），送入 consoleintr()。所以 USB 键盘和串口一样可以登录、输入命令、按 Ctrl-C。

这正是 Linux 的分层：硬件驱动只关心“产生了什么事件”，不关心“谁在读”。图 7.16 把这条分发路径画全：同一个 usbkbd_rx_work() 先经 input core 把键码发布成事件，再单独把字符送进控制台。

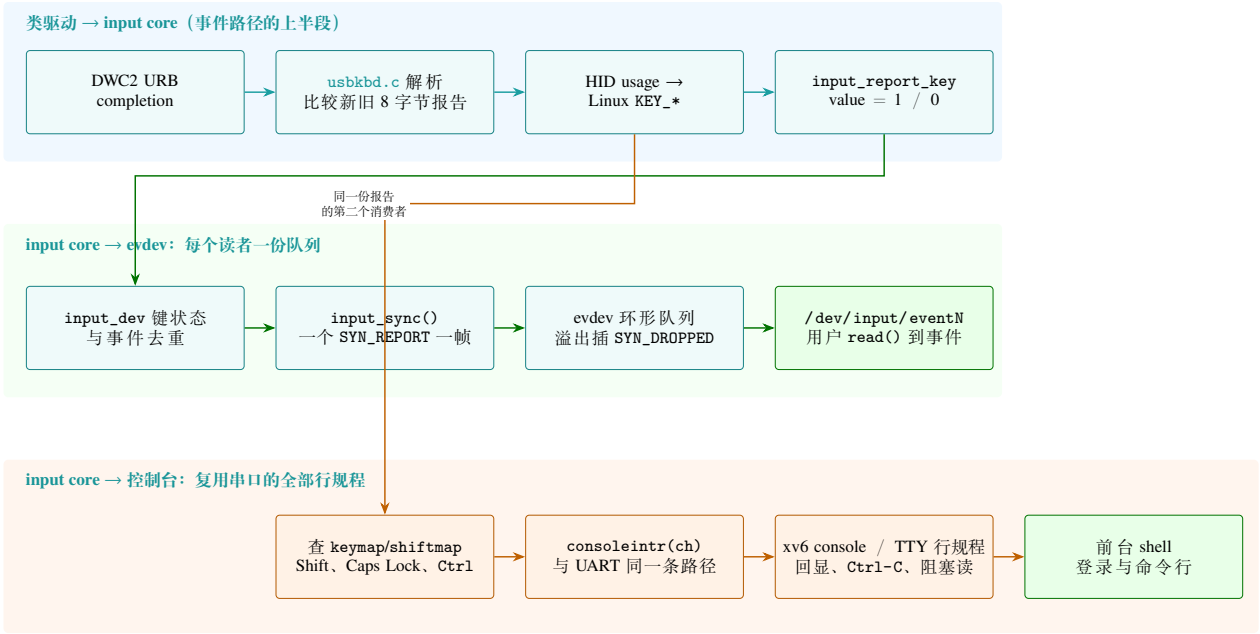


图 7.16: 一份 HID 报告进入输入子系统后的两条出路。上面两行是事件路径：类驱动解析报告、转成 Linux 键码，由 input core 去重、成帧，再复制到每个 open 自己的 evdev 队列；最后一行是字符路径：同一份报告查表转成字符后交给 consoleintr()，与 UART 收到的字节共用同一套行规程。两条路径互不依赖，因此读 /dev/input/eventN 的程序和登录会话可以同时存在，谁也不会把按键“读走”

内核侧的按键序列. usbkbd_rx_work() 处理一份报告时的动作顺序值得摊开看，因为它同时喂了上面两个处理者：

1. 本次 report[2..7] 里有、previous[2..7] 里没有的 usage 是新按下：先 input_report_key(kbd->input, hid_to_key[code], 1)，再查 keymap/shiftmap 生成字符；
2. 上次有、本次没有的 usage 是松开，input_report_key(..., 0)；usage 1 ~ 3 是错误码，两端都跳过；